Godling Agent Implementation Guide

1. 文档目标

本文件是 `Godling` 项目的实施手册，面向后续执行开发任务的编码 agent 或工程师。

项目需求的唯一来源是以下三个策划文件：

- `战斗系统.xlsx`
- `大地图系统设计.xlsx`
- `物品表.xlsx`

本手册的目标不是逐页复述策划，而是将三份表格收敛为一套可以直接落地到 `Godot 4.x + GDScript` 的

- 核心玩法闭环是什么
- MVP 必须先做什么
- 每个系统的职责边界是什么
- 数据应如何组织
- 开发顺序和验收标准是什么

==========

2. 游戏定义

2.1 核心定位

`Godling` 是一款从《苏丹的游戏》和类《鸭科夫》搜打撤体验中获取灵感的 2D 单局回合制搜打撤游戏。

玩家进入一张 2D 地图后，不是自由大地图跑图，而是在单局内按回合推进。每一回合，地图上会刷新一组可选事

- 战斗事件
- 随机事件
- 叙事事件

玩家通过这些事件逐步获得：

- 物资
- 藏品
- 货币
- 剧情推进
- 系统解锁

随着回合增加，风险同步升高。玩家需要在“继续深入获取更高收益”和“及时撤离保住当前战利品”之间做决策。

2.2 核心卖点

项目最重要的卖点不是纯战斗，而是“通过任务系统把叙事植入搜打撤循环”。

任务系统至少承担以下三项职责：

1. 给玩家明确目标，避免纯随机刷事件导致体验发散。
2. 让剧情随着局内推进自然出现，而不是独立于玩法之外的文本块。
3. 通过任务完成、剧情标记和解锁标记，逐步开放更多系统、更多事件和更深层内容。

因此，战斗系统是重要子系统，但在整体结构中应被视为“事件的一种执行形式”，而不是整个项目的唯一主轴。

==========

3. 顶层设计原则

3.1 先做单局闭环，再做复杂内容

MVP 的第一优先级不是做厚战斗，而是打通这条完整循环：

`进入地图 -> 刷新事件 -> 选择一个事件 -> 事件结算 -> 更新风险与资源 -> 决定继续或撤离 -> 最终结

如果这条循环不成立，单独完成 AI、掉落、任务文案或背包都没有意义。

3.2 所有内容必须数据驱动

三个 Excel 文件是策划源，不应在游戏运行时直接读取。运行时应只读取整理后的 JSON 数据。

以下内容必须配置化：

- 任务定义
- 事件定义
- 战斗定义
- 单位定义
- 战斗内事件定义
- 物品定义
- 掉落池定义
- 解锁标记
- 剧情标记
- 地图基础数值与成长参数

3.3 局内收益与局外成长分层

项目必须明确区分：

- 局内临时收益：本局中获得但尚未带出的物品、货币、藏品
- 局外永久收益：成功撤离后真正进入存档的资源
- 永久叙事成长：剧情标记、任务完成记录、系统解锁

默认规则：

- 局内死亡时，未撤离物品、货币、藏品丢失
- 剧情推进与系统解锁默认即时生效并永久保留

3.4 用标准化状态替代隐式逻辑

不要将任务完成、剧情推进、系统开放写成分散硬编码。所有关键状态必须落到可追踪字段：

- `story_flags`
- `unlock_flags`
- `completed_tasks`
- `map_stats`
- `danger_level`
- `run_loot`

==========

4. 技术基线

4.1 引擎与语言

- 引擎：`Godot 4.x`
- 语言：`GDScript`
- 目标：桌面端优先，键鼠交互优先

4.2 顶层状态

使用 Autoload 管理三个顶层状态：

`ContentDB`

职责：

- 加载全部 JSON 内容定义
- 建立 `id -> definition` 查询
- 提供按标签、类型、权重的检索能力
- 负责基础数据校验

`ProgressionState`

职责：

- 管理跨局永久存档
- 记录已完成任务、剧情节点、系统解锁、永久货币、永久仓库物品
- 为新开局提供基础可用内容

`RunState`

职责：

- 管理当前单局状态
- 记录当前地图、当前回合、当前危险度、当前事件列表、局内临时战利品、局内地图数值、当前装备快照、当前

==========

5. 推荐目录结构

在尚未存在 Godot 项目的前提下，建议按以下方式组织：

```
res://
  autoload/
    content_db.gd
    progression_state.gd
    run_state.gd
  data/
    items.json
    loot_tables.json
    units.json
    battle_events.json
    battles.json
    events.json
```

```
    tasks.json
    maps.json
    progression_defaults.json
  systems/
    world/
      world_director.gd
      event_generator.gd
      event_resolver.gd
      extraction_service.gd
    battle/
      battle_director.gd
      unit_controller.gd
      ai_system.gd
      combat_system.gd
      projectile_system.gd
      battle_event_system.gd
      result_resolver.gd
    inventory/
      inventory_service.gd
      equipment_service.gd
      reward_service.gd
  ui/
    world_map/
    battle/
    common/
  scenes/
    world_map.tscn
    battle_scene.tscn
    reward_panel.tscn
    inventory_panel.tscn
  tests/
```

实现时可以调整路径，但模块职责不要混淆。

==========

6. 核心数据模型

以下结构是 MVP 的稳定接口，后续实现应尽量围绕这些模型展开。

6.1 `ItemDef`

来自 `物品表.xlsx`。

建议字段：

```
{
  "id": "20001",
  "name": "民生物资",
  "type": 2,
  "quality": 1,
  "icon": "res://...",
  "price": 5,
  "description": "...",
  "tags": ["Military World", "Basic supplies"],
  "burden": 3,
  "auto_open": false,
  "code": ""
```

```
}
```

说明：

- `type=10` 表示藏品，可装备
- `type=5` 表示宝箱/掉落范围类物品
- `type=9` 表示 key 物品，可触发特殊事件

6.2 `LootTableDef`

来自 `物品表.xlsx` 的掉落池功能。

MVP 先支持两类能力中的第一类：

- 固定掉落
- 概率掉落

建议字段：

```
{
  "id": "ordinary_materials_1",
  "mode": "fixed_random",
  "entries": [
    { "kind": "item", "id": "10001", "count": 50, "weight": 10, "prob": 1.0 },
    { "kind": "item", "id": "60001", "count": 1, "weight": 5, "prob": 0.7 }
  ]
}
```

说明：

- `检索池随机` 作为第二阶段功能预留，不进入 MVP 核心交付
- 不做掉落池嵌套

6.3 `RewardPackage`

统一战斗、随机事件、叙事事件的奖励出口。

建议字段：

```
{
  "currencies": [{ "id": "10001", "count": 20 }],
  "items": [{ "id": "20001", "count": 2 }],
  "relics": [{ "id": "100001", "count": 1 }],
  "buffs": [{ "id": "80001", "count": 1 }],
  "loot_tables": [{ "id": "ordinary_materials_1", "rolls": 1 }],
  "unlock_flags": ["equipment_tab_enabled"],
  "world_effects": [
    { "stat": "government_reputation", "op": "add", "value": 2 }
  ]
}
```

规则：

- `currencies/items/relics` 先进入局内临时背包
- 只有撤离成功后才写入永久存档
- `unlock_flags/world_effects/story_flags` 立即生效

6.4 `EventDef`

这是修订版方案中的核心对象。地图上所有可选内容最终都要归一为 `EventDef`。

建议字段：

```
{
  "id": "event_trace_demon",
  "type": "battle",
  "title": "恶魔的行踪",
  "description": "调查恶魔的踪迹",
  "task_id": "task_trace_demon",
  "conditions": ["turn>2"],
  "danger_weight": 1.2,
  "costs": [],
  "rewards": { "...": "RewardPackage" },
  "on_success": [],
  "on_failure": [],
  "battle_id": "battle_trace_demon",
  "narrative": null,
  "random_resolution": null
}
```

三种 `type`：

- `battle`
- `random`
- `narrative`

规则：

- `battle` 通过 `battle_id` 进入战斗场景
- `random` 通过配置化效果在地图层即时结算
- `narrative` 展示叙事文本与分支，产出剧情和解锁

6.5 `TaskDef`

任务不是事件的替代品，而是事件的叙事包装层。

建议字段：

```
{
  "id": "task_trace_demon",
  "name": "恶魔的行踪",
  "task_type": "search",
  "pool_type": "probability",
  "base_weight": 10,
  "conditions": ["turn>2"],
  "weight_rules": [
    { "if": "map.demon_activity>5", "add": 5 }
  ],
  "completion_rule": "complete_event",
  "event_ref": "event_trace_demon",
  "battle_ref": "",
  "reward_preview": [{ "type": "loot_table", "id": "ordinary_materials_1" }],
  "effects": [
    { "stat": "government_reputation", "op": "add", "value": 2 }
  ],
  "story_flags": ["story_trace_demon_seen"],
```

```
  "unlock_flags": [],
  "map_ids": ["map_world_1_bridge"],
  "guarantee_counter": ""
}
```

职责:

- 决定任务是否能生成
- 决定在任务卡上显示什么
- 决定完成后修改哪些地图数值、剧情标记与解锁标记

6.6 `BattleDef`

来自 `战斗系统.xlsx` 的模板战斗。

建议字段:

```
{
  "id": "battle_trace_demon",
  "map_id": "map_world_1_bridge",
  "victory_type": "kill_target",
  "enemy_spawns": [
    {
      "unit_id": "enemy_weak_demon",
      "position": [20, 20],
      "count": 3,
      "summon_rules": [],
      "notify_rules": [],
      "response_rules": []
    }
  ],
  "map_events": ["battle_event_void_rift"],
  "hero_spawn": [10, 10],
  "ally_spawns": []
}
```

6.7 `BattleEventDef`

来自 `战斗系统.xlsx` 的战斗内事件模板。

建议字段:

```
{
  "id": "battle_event_void_rift",
  "event_type": "summon",
  "trigger": ["elapsed>=30"],
  "frequency": 30,
  "notify": {
    "text": "恶性空间裂缝即将打开",
    "countdown": 10,
    "hint": ""
  },
  "response": {
    "type": "end",
    "keyword": "aether",
    "level": 1,
    "time_limit": 10
  },
```

```
  "payload": {}
}
```

6.8 `UnitDef`

来自 `战斗系统.xlsx` 的单位数据和模板单位。

建议字段：

```
{
  "id": "enemy_weak_demon",
  "camp": "enemy",
  "name": "游荡恶魔",
  "move_speed": 10,
  "move_type": "walk",
  "size": 5,
  "hp": 20,
  "attack": {
    "type": "melee",
    "power": 2,
    "speed": 1.0,
    "range": 2,
    "cast_time": 0.3
  },
  "vision": 8,
  "movement_ai": "chase",
  "combat_ai": "melee_attack",
  "tags": ["demon", "melee", "shallow_void"],
  "skills": []
}
```

6.9 `RunState`

建议运行时字段：

```
{
  "run_id": "uuid",
  "map_id": "map_world_1_bridge",
  "turn": 1,
  "danger_level": 0,
  "available_events": [],
  "map_stats": {
    "development": 0,
    "public_reputation": 0,
    "government_reputation": 0,
    "demon_activity": 0,
    "fear": 0,
    "population": 0
  },
  "temporary_inventory": [],
  "temporary_currencies": [],
  "temporary_relics": [],
  "equipped_relics_snapshot": [],
  "completed_tasks": [],
  "story_flags_gained_this_run": [],
  "unlock_flags_gained_this_run": [],
  "can_extract": false,
  "is_dead": false,
```

```
  "is_extracted": false
}
```

6.10 `ExtractionResult`

建议结算字段：

```
{
  "status": "extracted",
  "saved_items": [],
  "saved_currencies": [],
  "saved_relics": [],
  "lost_items": [],
  "completed_tasks": [],
  "story_flags_applied": [],
  "unlock_flags_applied": [],
  "map_stat_changes": []
}
```

==========

7. 单局核心循环

这是项目的最重要实现目标。任何开发工作都应该服务于这个循环。

7.1 开局

玩家从局外状态进入一张地图时：

1. `RunState` 初始化
2. 从 `ProgressionState` 读取永久状态
3. 从永久仓库读取已装备的藏品，生成局内装备快照
4. 读取地图基础数值和可用任务池
5. 生成首回合事件

7.2 回合刷新

每回合默认生成 3 个普通事件位。

额外事件不占普通位：

- 任务线事件
- key 物品触发事件
- 保底计数器触发事件

每回合玩家只能选择并处理 1 个事件。

7.3 事件处理

统一由 `EventResolver` 驱动。

`battle` 路径

流程：

1. 进入 `BattleScene`
2. 根据 `BattleDef` 生成敌人、友方、英雄和地图事件
3. 战斗自动进行
4. 根据战斗结果返回地图层
5. 应用奖励、任务完成、剧情推进和失败惩罚

`random` 路径

流程：

1. 地图层直接弹出事件面板
2. 显示文案、成本、收益、可能损失
3. 玩家确认后直接执行
4. 修改 `RunState` 和相关地图数值

`narrative` 路径

流程：

1. 显示叙事文本和选项
2. 根据选项产出不同 `story_flags`、`unlock_flags`、地图效果和奖励
3. 将结果写回状态

7.4 回合结束

事件处理完毕后：

1. 回合数 `turn + 1`
2. `danger_level` 上升
3. 地图数值按规则变化
4. 更新任务权重和事件池
5. 刷新下一回合候选事件
6. 若玩家已经完成至少一个事件，则允许撤离

7.5 撤离或死亡

撤离

- MVP 中撤离默认必定成功
- 结算本局临时背包
- 将可带出资源写入 `ProgressionState`
- 写入剧情、解锁、任务完成记录

死亡

- 局内临时物品、货币、藏品丢失
- 剧情标记与系统解锁默认保留
- 结束本局

==========

8. 世界层系统拆分

8.1 `WorldDirector`

职责：

- 驱动单局回合流程
- 控制开局、回合推进、事件刷新、撤离和结算
- 协调 `EventGenerator`、`EventResolver` 与 UI

8.2 `EventGenerator`

职责：

- 从任务池和事件池中挑选本回合候选项
- 处理常规池、概率池、任务线、保底计数器、key 物品触发
- 根据回合数、危险度、地图数值和剧情标记修正权重

最低规则：

- 每回合 3 个普通事件
- 优先过滤不可出现的事件
- 普通任务按权重随机抽取
- 额外事件单独附加

8.3 `EventResolver`

职责：

- 根据事件类型选择结算路径
- 统一产出结构化结算结果
- 将结果回写到 `RunState`

8.4 `ExtractionService`

职责：

- 处理撤离成功和死亡失败的差异化结算
- 将局内临时资源转换为局外永久资源
- 生成 `ExtractionResult`

==========

9. 战斗系统拆分

战斗是重要模块，但在产品结构里属于事件执行器。

9.1 `BattleDirector`

职责：

- 接收 `BattleDef`
- 初始化战斗场景
- 驱动单位更新、事件更新和胜负判定

9.2 MVP 战斗范围

MVP 只实现以下内容：

- 三类单位：敌人、友方、英雄
- 单位不可重叠
- 两种移动方式：步行、飞行
- 两种攻击方式：近战、远程平射
- 基础自动索敌与追杀 AI
- 基础召唤/buff 战斗事件
- 战斗胜负与结果回传

明确不进 MVP 第一阶段的内容：

- 抛射逻辑
- 墙体破坏
- 搜寻、巡航、保卫等复杂移动 AI
- 玩家拖拽选点施法
- 高级战斗脚本编排

9.3 子系统职责

`UnitController`

- 管理单个单位属性、状态、受伤、死亡

`AISystem`

- 处理基础索敌、追杀、攻击触发

`CombatSystem`

- 处理伤害结算、攻击频率、攻击距离、命中

`ProjectileSystem`

- 仅支持 MVP 的基础平射

`BattleEventSystem`

- 管理战斗内事件的触发、频率、通知、响应、结束

`ResultResolver`

- 将胜负、损失、奖励、完成任务等输出给地图层

==========

10. 物品、背包与装备

10.1 物品层级

依据 `物品表.xlsx`，MVP 中至少要覆盖：

- 资源
- 普通物品
- 宝箱/掉落范围
- 消耗品
- key 物品
- 藏品
- 临时 buff

10.2 局内临时背包

局内临时背包用于存放本局尚未撤离的收益：

- 事件奖励
- 战斗掉落
- 随机事件所得

这些资源只在撤离成功后才进入永久仓库。

10.3 永久仓库

永久仓库存放已带出的物品，不设容量上限。

MVP 范围：

- 查看
- 按类型/标签筛选
- 提供装备来源

不做：

- 家园系统联动
- 复杂跳转
- 图鉴

10.4 装备系统

规则：

- 仅 `type=10` 的藏品允许装备
- 开局时从永久仓库读取当前装备
- 进入战斗事件时把装备转换成战斗被动效果

MVP 默认开放槽位数量可先写死。

10.5 `InventoryService`

职责：

- 局内获得物品
- 局内移除物品
- 撤离时转存
- 死亡时清算

10.6 `EquipmentService`

职责：

- 校验是否可装备
- 读取/写入装备栏
- 生成战斗加成快照

==========

11. 任务、叙事与解锁

11.1 任务系统是叙事骨架

任务需要完成以下绑定：

- 任务卡上的目标展示
- 与事件的直接关联
- 与地图数值和后续权重的关联
- 与剧情推进和系统开放的关联

任务完成后至少允许做四类事：

- 发放奖励
- 修改地图数值
- 写入剧情标记
- 写入解锁标记

11.2 `story_flags`

用途：

- 控制剧情体验顺序
- 控制叙事事件能否出现
- 控制后续任务是否开放

11.3 `unlock_flags`

MVP 中明确接入以下 3 个标准化开关：

- `narrative_events_enabled`
- `equipment_tab_enabled`
- `special_item_event_enabled`

要求：

- UI 是否显示相关页签必须读这个标记
- 特殊事件是否能生成必须读这个标记
- 后续任何新系统都应以增加 `unlock_flag` 的方式扩展，而不是新增硬编码分支

11.4 文本与叙事展示

MVP 中叙事事件采用配置化文本和简单分支，不引入脚本语言。

最小能力：

- 标题
- 正文

- 1 到 3 个选项
- 每个选项对应不同结果

==========

12. 风险收益曲线

这是搜打撤体验是否成立的关键。

12.1 必须存在的风险增长

每回合结束后，至少同步提升：

- `danger_level`
- 敌对事件权重
- 稀有事件权重
- 掉落系数或奖励期望

12.2 必须可感知的收益增长

随着回合提高，玩家应感知到：

- 更高价值奖励
- 更高概率接触稀有任务
- 更深层剧情与系统开放

12.3 设计要求

如果风险只增长而收益不增长，玩家会倾向于尽快撤离。

如果收益只增长而风险不增长，撤离时机就失去意义。

因此：

- 回合数必须同时影响难度和奖励
- 任务权重调整必须明确反映“继续深入”的机会成本和收益

==========

13. 首批内容范围

为了保证可交付，MVP 首批内容建议压缩为一张地图、少量任务和少量战斗模板。

13.1 地图

- 1 张基础地图

13.2 任务

建议优先整理以下任务进入首批内容：

- `民生物资（深入）`
- `恶魔的行踪`

- `控制缓冲区恶魔数量`

理由：

- 能覆盖搜寻、击杀、权重变化等不同任务类型
- 能覆盖常规与概率池
- 能验证任务推进对地图数值的影响

13.3 战斗模板

建议优先接入：

- `隐匿的追猎魔`
- `肃清城北郊区`

13.4 单位

建议首批最少包含：

- 游荡恶魔
- 活化机械
- 野犬残形
- 英雄 `a01`

13.5 事件

建议首批至少包含：

- 2 个战斗事件
- 2 个随机事件
- 2 个叙事事件

13.6 物品

建议首批至少整理：

- 货币
- 民生物资
- 战术绷带
- 恶魔榨取物
- 1 个 key 物品
- 1 到 2 个藏品

==========

14. 分阶段开发顺序

Phase 1：工程底座与数据落盘

目标：

- 建立 Godot 工程结构
- 建立 Autoload
- 将 Excel 内容整理为 JSON
- 完成 `ContentDB` 加载与校验

验收：

- 启动时可以读取全部 JSON
- 缺失字段、重复 `id`、坏引用会报错

Phase 2：单局地图循环

目标：

- 开局
- 事件刷新
- 事件选择
- 回合推进
- 撤离入口

验收：

- 玩家可以连续进行至少 3 个回合
- 回合之间能刷新不同任务
- 第 1 次事件完成后出现撤离按钮

Phase 3：事件结算框架

目标：

- `battle/random/narrative` 三类事件统一接入 `EventResolver`
- 随机事件与叙事事件可直接完成结算

验收：

- 三种事件都能从地图选择并正确返回结果

Phase 4：战斗 MVP

目标：

- 实现基础自动战斗
- 支持基础战斗内事件
- 返回战斗结果到地图层

验收：

- 模板战斗可运行
- 单位会移动、索敌、攻击、死亡
- 战斗胜败能影响后续状态

Phase 5：背包、装备与最终结算

目标：

- 局内临时背包
- 局外永久仓库
- 装备栏
- 撤离结算与死亡清算

验收：

- 撤离后物品进入永久存档
- 死亡后临时物品清空
- 藏品可装备并影响战斗

Phase 6：叙事驱动与系统解锁

目标：

- 接入 `story_flags`
- 接入 `unlock_flags`
- 用任务推进开放叙事事件、装备页签、特殊道具事件

验收：

- 某个任务完成后，后续回合能出现新的叙事内容
- UI 页签与事件池能正确响应解锁

==========

15. 测试与验收标准

15.1 数据校验

必须验证：

- 重复 `id`
- 缺失必要字段
- 掉落池引用不存在物品
- 任务引用不存在事件
- 事件引用不存在战斗
- 条件表达式格式非法

15.2 单局循环

必须验证：

- 开局后能生成事件
- 每回合只能处理 1 个事件
- 普通事件刷新后不会无故保留
- 任务线/保底/key 事件按规则追加

15.3 撤离与死亡

必须验证：

- 撤离成功后临时资源全部转入永久存档
- 死亡后临时资源全部清空
- 剧情标记和系统解锁在死亡后仍保留

15.4 任务与叙事

必须验证：

- 任务卡能展示目标、影响、奖励预览
- 完成任务后能修改地图数值

- 完成任务后能解锁后续任务或叙事事件

15.5 战斗

必须验证：

- 单位不会重叠
- 基础索敌正常
- 近战和平射都能造成伤害
- 单位死亡后能正确移除
- 战斗事件的触发、通知、响应、结束能正常工作

15.6 风险曲线

必须验证：

- 回合增长后高危险任务出现概率上升
- 奖励期望同步增长
- 提前撤离与继续深入之间存在明显权衡

15.7 存档持久化

必须验证：

- 永久仓库保留
- 已完成任务保留
- 剧情标记保留
- 解锁标记保留
- 下一局事件池受这些状态影响

==========

16. 非目标与后续扩展

以下内容不进入第一阶段实施：

- 掉落池嵌套
- 检索池随机的完整实现
- 复杂合成系统
- 家园系统
- 图鉴/百科系统
- 高级跳转 UI
- 撤离战
- 复杂玩家手动施法
- 抛射与墙体破坏
- 高级脚本式战斗编排

推荐的第二阶段扩展顺序：

1. 检索池随机掉落
2. 更复杂的移动 AI
3. 主动技能拖拽施法
4. 剧情与事件合成
5. 非战斗事件链
6. 撤离遭遇战

==========

17. 执行约束

后续 agent 在实现本项目时，应遵守以下约束：

1. 任何新增系统都必须先判断它属于 `world / battle / inventory / progression / ui` 中哪一
2. 任何新内容优先写配置，再写逻辑，不要反过来。
3. 任何功能若无法进入单局闭环，应延后。
4. 若出现“战斗实现得很完整，但撤离、任务推进、叙事解锁还没通”的情况，优先补闭环，不继续加厚战斗。
5. 除非已经明确进入第二阶段，否则不要主动扩展到复杂掉落池、复杂 AI 或大而全 UI。

==========

18. 最终交付定义

MVP 完成时，必须能够稳定演示以下体验：

1. 玩家从局外进入一张地图。
2. 首回合出现一组任务/事件卡。
3. 玩家选择一个事件并完成它。
4. 若事件为战斗，则进入自动战斗并带回结果。
5. 若事件为随机/叙事，则直接在地图层结算。
6. 玩家在多个回合中逐步积累收益与风险。
7. 玩家可以在完成至少一个事件后选择撤离。
8. 撤离成功后，本局带出的物品进入永久仓库。
9. 已完成任务、剧情标记和系统解锁影响下一局。

只有当以上 9 点全部成立时，才算真正完成本手册定义的第一阶段目标。